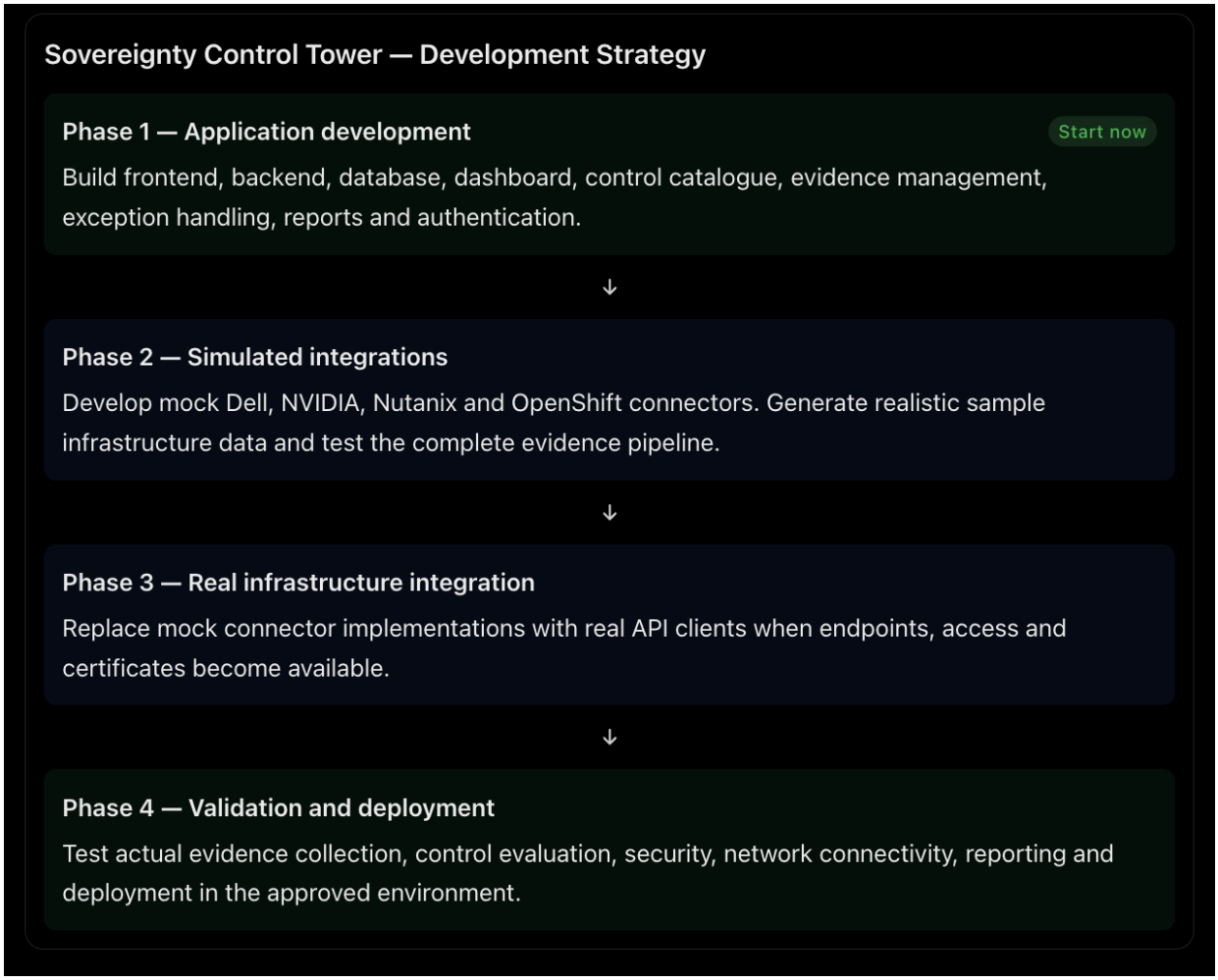
Sovereignty Control Tower application

approach: build the complete Sovereignty Control Tower application independently, using mock data and simulated integrations initially. Once the infrastructure team provides endpoints and credentials, we connect the real hardware through integration adapters without rebuilding the application.

The key principle is:

Build the application first. Integrate the infrastructure progressively.

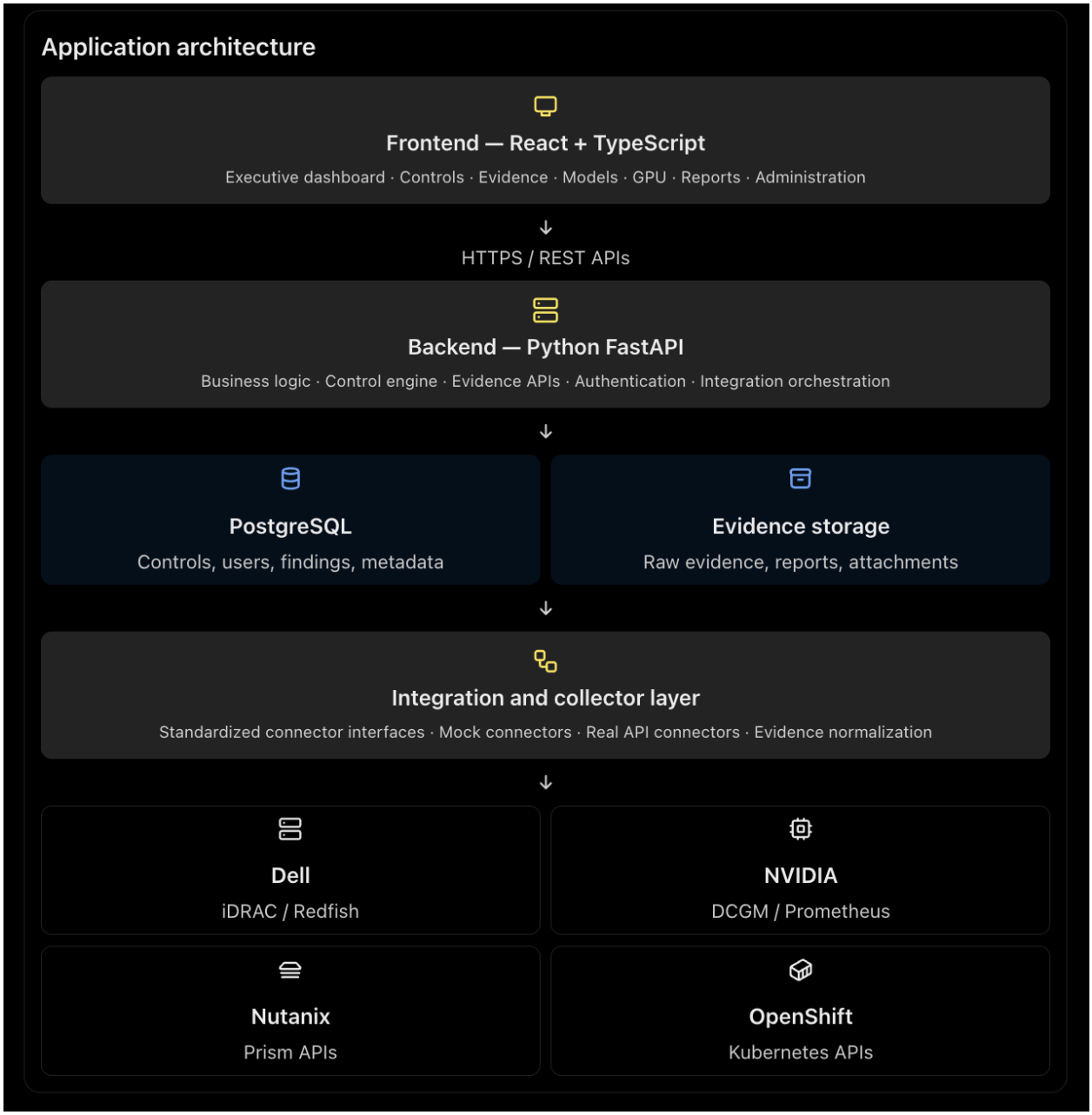
1. Our development approach



2. What exactly are we going to build?

We should treat this as a full-stack enterprise application, not just a dashboard.

I recommend the following architecture:



3. The most important design decision: Build connectors independently

We should never write Dell-specific or NVIDIA-specific logic directly inside the dashboard or control evaluation engine.

Instead, create a common integration interface.

For example, suppose we want GPU information.

The application should request:

```
None
gpu_connector.get_inventory()gpu_connector.get_health()gpu_connector.get_metrics()
```

Initially, these functions will use mock data.

Later, the implementation will call NVIDIA DCGM Exporter and Prometheus.

The frontend and the rest of the backend will not need to know whether the data came from a mock connector or real hardware.

The same approach applies to Dell:

```
None
dell_connector.get_servers()dell_connector.get_hardware_health()dell_connector.get_firmware_inventory()
```

And Nutanix:

```
None
nutanix_connector.get_clusters()nutanix_connector.get_hosts()nutanix_connector.get_virtual_machines()
```

And OpenShift:

```
None
openshift_connector.get_nodes()openshift_connector.get_namespaces()openshift_connector.get_workloads()
```

This is what will make our application integration-ready from day one.

4. How the application will behave without real endpoints

We will create a dedicated integration configuration page.

For every integration,

Integration	Initial mode	Future mode
Dell iDRAC	Mock	Real Redfish API
NVIDIA GPU	Mock	DCGM + Prometheus
Nutanix Prism	Mock	Real Prism API
OpenShift	Mock	Kubernetes API
Keycloak	Local development auth	Enterprise OIDC
Vault	Development secrets	Enterprise Vault

Each connector should have:

- Connection status.
- Integration mode: Mock or Live.
- Last successful collection.
- Last error.
- Collection frequency.
- Number of collected records.
- Test connection action.
- Enable/disable action.

Important: the application must clearly distinguish simulated data from live infrastructure data. A mock GPU showing 70% utilization must never be represented as an actual GPU measurement.

5. What modules should we develop first?

I recommend this development sequence.

Priority	Module	What we build
1	Application foundation	Login, layout, navigation, environment selectors
2	Executive Overview	KPIs, sovereignty dimensions, status cards
3	Sovereignty Dimensions	All eight dimensions and their control mappings
4	Control Catalogue	Controls, requirements, evidence mapping
5	Evidence Repository	Upload, metadata, hashes, validation
6	Exceptions & Remediation	Findings, owners, severity, deadlines
7	Infrastructure & GPU	Dell, NVIDIA, Nutanix and OpenShift views
8	AI Governance	Models, RAG provenance, AI workload inventory
9	Evidence Collectors	Connector management and collection history
10	Assurance Reports	Assessment summaries and exports

11	Approvals	Review workflows and audit trail
12	Administration	Users, roles, policies and configurations

We should retain the eight sovereignty dimensions:

1. Governance
2. Data
3. Intelligence
4. Compute
5. Operations
6. Network
7. Supply Chain
8. Continuity

The final control count and control wording should come from the team's validated control baseline, not an assumed number.

6. Suggested project structure

I recommend a modular monolith initially. It will be simpler to develop, test and deploy than starting with multiple microservices. We can split components later if needed.

```
None
sovereignty-control-tower/
|
├── frontend/
|   ├── src/
|   |   ├── components/
|   |   ├── layouts/
|   |   ├── pages/
|   |   |   ├── overview/
|   |   |   ├── dimensions/
|   |   |   ├── controls/
|   |   |   ├── evidence/
|   |   |   ├── exceptions/
|   |   |   ├── infrastructure/
|   |   |   ├── models/
|   |   |   ├── reports/
|   |   |   └── administration/
|   |   ├── services/
|   |   ├── hooks/
|   |   └── types/
|   └── package.json
|
```

```
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── core/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   ├── repositories/
│   │   ├── integrations/
│   │   │   ├── base/
│   │   │   ├── mock/
│   │   │   ├── dell/
│   │   │   ├── nvidia/
│   │   │   ├── nutanix/
│   │   │   └── openshift/
│   │   ├── evidence/
│   │   └── main.py
│   └── requirements.txt
│
├── database/
│   ├── migrations/
│   └── seed_data/
│
├── infrastructure/
```

```
|   ├── docker/
|   ├── kubernetes/
|   └── helm/
|
├── tests/
└── README.md
```

7. Our first development milestone

Rather than trying to implement every module at once, I would target a working application foundation in the first 10 working days.

10-day development plan

Day 1 — Project setup
React, FastAPI, PostgreSQL, Git repository and development environment.

Day 2 — Dashboard UI
Sidebar, top navigation, environment selection and responsive layout.

Day 3 — Sovereignty framework
Eight dimensions, control catalogue, database schema and seed data.

Day 4 — Evidence management
Evidence ingestion, metadata, file storage and SHA-256 hashing.

Day 5 — Control evaluation
Control status calculation, evidence mapping and assessment history.

Day 6 — Exceptions and approvals
Findings, remediation workflow, owners and approval records.

Day 7 — Infrastructure modules
Dell, GPU, Nutanix and OpenShift views populated by mock connectors.

Day 8 — AI governance
Model inventory, RAG provenance and workload visibility.

Day 9 — Collector management and reporting
Integration configuration, collection logs and assurance reports.

Day 10 — End-to-end demonstration
Demonstrate simulated infrastructure collection through to control assessment

8. What I need us to decide before coding

Area	My recommendation
Frontend	React + TypeScript
UI styling	Tailwind CSS
Backend	Python + FastAPI
Database	PostgreSQL
Evidence storage	S3-compatible object storage
Authentication	Keycloak-ready architecture
Infrastructure integration	Independent Python connectors
GPU monitoring	DCGM + Prometheus
Hardware API	Dell Redfish
Deployment	Docker initially, OpenShift/Helm later
Development strategy	Mock first, real integrations later

One additional architectural rule: we should design the application to run locally during development, then package it for deployment inside the approved sovereign environment.